Cletus — Section 1, Day 4

- **Date:** [YYYY-MM-DD]
- **Course:** Week 1, Day 4 — Transformers, parameters, tokens, the illusion of memory, context windows, API costs
- **Lab notebook:** week1/day4.ipynb
- **Time spent:** [e.g. 2h]

Key concepts

- **Transformer** — the architecture behind GPT and almost every modern LLM, from Google's 2017 paper *Attention Is All You Need*. Older models (RNNs and LSTMs) read text one word at a time. *Attention* lets the model look at all the words at once and weigh which ones matter, so it can be trained in parallel at huge scale.
- **Emergent intelligence** — at large scale, models show abilities nobody explicitly trained into them.
- **Parameters (weights)** — the numbers a model learns during training; they measure its "size". GPT-1 had ~117M → GPT-3 175B → Llama 3.1 up to 405B → DeepSeek V3/R1 671B. Top closed models are thought to have trillions.
- **Tokens** — the chunks of text a model reads and writes; they're neither letters nor whole words. Rule of thumb for English: **1 token ≈ 4 characters ≈ ¾ of a word**, so 1,000 tokens ≈ 750 words. Common words are 1 token, while rare words and names split into several. The leading space is often part of the token (`" pie"`).
- **Tokenizer** — turns text into token IDs and back. Each model family has its own; `tiktoken` is OpenAI's.
- **Stateless calls / illusion of memory** — every API call starts fresh. The model only "remembers" because we resend **the whole conversation** every time, including its own earlier replies (`assistant` role). ChatGPT works exactly like this.
- **Context window** — the most tokens a model can handle in one call: system prompt + whole conversation + its reply. Very long chats eventually hit this limit.
- **API costs** — charged per token (usually priced per million). Input and output are priced separately, and output costs more. Resending the conversation means paying for it again every turn.

Code / examples

See the tokens:

```
import tiktoken

encoding = tiktoken.encoding_for_model("gpt-4.1-mini")
tokens = encoding.encode("Hi my name is Ed and I like banoffee pie")
for token_id in tokens:
    print(token_id, "=", repr(encoding.decode([token_id])))
print(len(tokens), "tokens")
```

"Memory" = resending the history:

```
messages = [
    {"role": "system", "content": "You are a helpful assistant"},
    {"role": "user", "content": "Hi! I'm Ed!"},
    {"role": "assistant", "content": "Hi Ed! How can I assist you today?"},
    {"role": "user", "content": "What's my name?"},
```

```
]
# Send only the last user message and the model can't know the name.
# Send the full history and it answers "Ed".
```

Exercise / challenge

1. Tokenize your full name, a sentence in another language, and a line of code. Which uses the most tokens per word?
2. Build a mini chatbot that keeps memory by adding every turn to `messages`:

```
from openai import OpenAI
openai = OpenAI()

messages = [{"role": "system", "content": "You are a helpful assistant"}]
while True:
    user = input("You: ")
    if user == "quit":
        break
    messages.append({"role": "user", "content": user})
    reply = openai.chat.completions.create(model="gpt-4.1-nano",
messages=messages).choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    print("AI:", reply)
```

3. After a few turns, count the tokens in all of `messages` and watch the cost grow with each turn.

Questions & blockers

Watch out for: - Models are bad at counting letters because they see tokens, not letters. - Long chats cost more each turn and can eventually go past the context window.

To discuss with my partner

- How could we keep a long chat cheap? For example, summarize older turns, or keep only the last N messages.
- Explain attention to each other in one sentence.

Resources

- OpenAI tokenizer playground: https://platform.openai.com/tokenizer
- *Attention Is All You Need*: https://arxiv.org/abs/1706.03762
- Model prices: https://openai.com/api/pricing